

Student Information

Full Name	Kaburu Morris
Reg No	C026-0034-2016
Submitted	2026-07-23 05:03:23

Overall Result

1/28 (3.6%) Grade: F

Score by Section

Section	Score	Pct
Activation Functions	1/5	20.0%
Weight Initialization	0/3	0.0%
Loss Functions	0/3	0.0%
Optimizers	0/4	0.0%
Backpropagation	0/3	0.0%
Regularization	0/4	0.0%
Batch Normalization	0/2	0.0%
Architecture Design	0/4	0.0%

Questions to Review

Q01. [Activation Functions]

What is the output range of the Sigmoid activation function?
Sigmoid = 1/(1+e^{-z}). As z->+inf output->1, as z->-inf output->0. It never reaches 0 or 1, so the range is the open interval (0, 1).

Q02. [Activation Functions]

Which activation function suffers from the 'Dying ReLU' problem?
ReLU outputs 0 for all z<=0 and its gradient is also 0 there. A neuron stuck at 0 stops receiving gradient updates and dies. Leaky ReLU fixes this with a small slope (0.01) for negative inputs.

Q03. [Activation Functions]

Why is Tanh preferred over Sigmoid for hidden layers?
Tanh outputs range (-1, 1) centered around 0. Sigmoid outputs (0, 1) are always positive, causing zig-zag gradient updates. Zero-centering avoids this inefficiency.

Q04. [Activation Functions]

What does the derivative of ReLU equal for z = 3.5?
ReLU derivative = 1 if z > 0, else 0. Since z=3.5 > 0, the derivative is exactly 1.

Q06. [Weight Initialization]

What problem occurs if you initialize ALL weights to zero?

With all weights=0, every neuron computes the same output and gets the same gradient. They update identically forever and symmetry is never broken.

Q07. [Weight Initialization]

Which initializer is recommended when using ReLU activations?

He initialization: $\text{std} = \sqrt{2/\text{fan_in}}$. The factor 2 compensates for ReLU zeroing roughly half the neurons. Xavier uses $\sqrt{2/(\text{fan_in} + \text{fan_out})}$, designed for Tanh/Sigmoid.

Q08. [Weight Initialization]

What does 'fan_in' mean in the context of weight initialization?

fan_in = number of inputs to a layer (input dimension). fan_out = number of outputs. Xavier uses both; He uses only fan_in.

Q09. [Loss Functions]

Which loss function should you use for BINARY classification?

Binary Cross-Entropy = $-[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$. Designed for 2-class problems with a Sigmoid output.

Q10. [Loss Functions]

A model predicts $p=0.95$ for a sample with true label $y=1$. What is the Binary Cross-Entropy loss?

$BCE = -\log(0.95) \approx -0.051$. A confident correct prediction gives a very small loss. If $p=0.05$ for $y=1$, $\text{loss} = -\log(0.05) \approx 3.0$.

Q11. [Loss Functions]

Why do we clip predictions before computing cross-entropy loss (e.g. clip to $1e-12$, $1-1e-12$)?

$\log(0) = -\infty$, making the loss undefined (NaN). Clipping keeps $\log()$ finite and numerically stable.

Q12. [Optimizers]

What does the momentum term do in SGD with Momentum?

Momentum: $v = \beta v + \eta \cdot \text{grad}$. Accelerates movement in consistent directions and dampens oscillations. Typical $\beta=0.9$ retains 90% of previous velocity.

Q13. [Optimizers]

Adam optimizer combines which two techniques?

Adam maintains m (first moment, like momentum) and v (second moment, like RMSProp). Update: $\theta \leftarrow \theta - \eta \cdot m_hat / (\sqrt{v_hat} + \epsilon)$.

Q14. [Optimizers]

Which optimizer typically converges fastest on small datasets?

Adam adapts the learning rate per-parameter using gradient history, making it robust to different gradient scales.

Q15. [Optimizers]

What is the purpose of bias correction in Adam (dividing by $1 - \beta^t$)?

At $t=1$, m is tiny because it started at 0. Dividing by $(1-\beta^t)$ scales it back to the true gradient magnitude.

Q16. [Backpropagation]

What does the chain rule tell us in backpropagation?

Chain rule: $dL/dW1 = (dL/da2) \cdot (da2/dz2) \cdot (dz2/da1) \cdot (da1/dz1) \cdot (dz1/dW1)$. Each term is a local gradient multiplied going backward.

Q17. [Backpropagation]

What is the vanishing gradient problem?

Sigmoid derivative max = 0.25. In a 10-layer net, gradient shrinks to $0.25^{10} \approx 0.000001$ at layer 1. ReLU derivative = 1 for $z > 0$, so gradients do not shrink.

Q18. [Backpropagation]

In a gradient flow chart, what does a gradient ratio greater than 1000x between layers indicate?

1000x difference means early layers receive almost no learning signal. Fix: use ReLU, BatchNorm, or residual connections.

Q19. [Regularization]

What is overfitting?

Overfitting = low train loss but high val loss. The model memorizes noise instead of learning general patterns.

Q20. [Regularization]

How does Dropout prevent overfitting?

With dropout rate=0.5, each neuron is randomly off 50% of training time. This prevents co-adaptation. At inference time, dropout is OFF.

Q21. [Regularization]

L2 regularization adds lambda multiplied by the sum of squared weights to the loss. What effect does this have?

*L2 adds $\lambda * W$ to each gradient update, decaying weights toward 0 but never exactly 0. Also called weight decay.*

Q22. [Regularization]

Which regularization setup produces the SMALLEST train/val loss gap?

L2 + Dropout combines weight penalty and random deactivation, providing stronger regularization than either alone.

Q23. [Batch Normalization]

What does Batch Normalization do to its input?

*BN: $x_{\text{norm}} = (x - \mu) / \sqrt{\text{var} + \text{eps}}$, output = $\gamma * x_{\text{norm}} + \beta$. γ and β are learned. Reduces internal covariate shift.*

Q24. [Batch Normalization]

Why does BatchNorm behave differently during training versus inference?

Training: μ and var from current mini-batch. Inference: uses stable running_mean and running_var (EMA). Needed because test batch size may be just 1.

Q25. [Architecture Design]

Why does the output layer use Sigmoid and NOT ReLU for binary classification?

Binary cross-entropy expects predictions in (0,1). Sigmoid squashes any real number to (0,1). ReLU could output 5.3 which has no probabilistic meaning.

Q26. [Architecture Design]

What is the purpose of using mini-batches during training with batch size 32?

Full-batch GD: exact gradient but slow. SGD (batch=1): fast but noisy. Mini-batch (32-256): balances speed and gradient quality.

Q27. [Architecture Design]

Early stopping uses patience=30. What does this mean?

No improvement for 30 consecutive epochs triggers a stop and restores the weights from the best epoch seen.

Q28. [Architecture Design]

Why can a single layer with no hidden layers NOT solve the XOR problem?

XOR: $(0,0) \rightarrow 0$, $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$, $(1,1) \rightarrow 0$. No straight line separates the 0s from the 1s.

A hidden layer creates a non-linear transformation making classes separable.

